

Publishing the Odometry Topic

1. Experiment Objective

In this experiment, you will learn how to publish the ROS2 standard `nav_msgs/msg/Odometry` odometry message through micro-ROS, while also fusing a `cmd_vel` speed subscription to implement a complete chassis control and state feedback node. Through this example, you will master motor encoder data acquisition, Dead Reckoning, ROS time synchronization, and the combined use of publishers and subscribers. This example publishes the topic `/odom_raw` at 10 Hz and simultaneously subscribes to `/cmd_vel` to control the motion of the car.

2. Experiment Principle

1. Keyword Explanation

Concept	Description
Odometry	The ROS2 standard odometry message type, defined in <code>nav_msgs/msg/Odometry</code> , containing position, orientation, linear and angular velocities, as well as their respective covariance matrices. It is the fundamental data source for robot localization and navigation
Dead Reckoning	A method of estimating the robot's position and orientation by integrating sensor data such as encoders. Starting from a known initial position, the current position is continuously estimated by integrating velocity and angular velocity over time, but the error accumulates over time
frame_id	A string in ROS2 used to identify coordinate frames. The <code>frame_id</code> in the message header indicates the coordinate frame in which the data is expressed. In this example, <code>odom_frame</code> is the world coordinate frame and <code>base_footprint</code> is the robot body coordinate frame
Covariance matrix	A mathematical tool representing the uncertainty of an estimate. In the Odometry message, <code>pose.covariance</code> describes the uncertainty of position and orientation, and <code>twist.covariance</code> describes the uncertainty of velocity. The smaller the value, the higher the confidence
Encoder	An angle sensor mounted on the motor shaft, which measures the rotation angle and direction of the motor through pulse counting. Combined with the wheel diameter, it can estimate the distance and speed of wheel movement, and is the core input device for Dead Reckoning
ROS time synchronization	The micro-ROS node synchronizes its clock with the ROS2 Agent through <code>rmw_uros_sync_session()</code> , obtaining a unified ROS timestamp and ensuring that data from different nodes is consistent and comparable on the time axis

2. Technical Architecture

This example implements a complete chassis control and state feedback node, serving as both a **publisher** and a **subscriber**.

Configuration Item	Value	Description
Node name	odom_publisher	ROS2 node name
Published topic	odom_raw	Topic for publishing odometry data
Message type	nav_msgs/msg/Odometry	ROS2 standard odometry message
Publish frequency	10 Hz	Period 100 ms
Subscribed topic	cmd_vel	Speed control topic
cmd_vel timeout	2000 ms	Automatically stops the car if no cmd_vel message is received for more than 2 seconds (different from the 1500 ms in Lesson 6)
QoS mode	Uplink best_effort (odometry), downlink reliable (cmd_vel)	The odometry uses best-effort transmission, cmd_vel uses reliable transmission

In terms of software architecture, the program creates 1 timer (publishes odometry at 10 Hz) and 1 subscriber (cmd_vel) simultaneously in micro_ros_task, uniformly scheduled by the executor (EXECUTOR_HANDLE_COUNT=2), ensuring the coordinated operation of communication and control.

3. Core Topic Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
/odom_raw	nav_msgs/msg/Odometry	best_effort	Odometry data (position + orientation + velocity), published at 10 Hz

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/msg/Twist	Reliable	linear.x (linear velocity m/s)	[-1.0, 1.0]	+= forward, -= backward
			angular.z (angular velocity rad/s)	[-6.0, 6.0]	+= turn left, -= turn right

Timeout safety: If no new cmd_vel is received within 2000 ms, the car stops automatically.

Usage Example:

```
# View the odometry data
ros2 topic echo /odom_raw
# Control the car to move forward
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}"
```

4. Middleware Configuration

micro-ROS uses a **static memory model** (`RCUTILS_AVOID_DYNAMIC_ALLOCATION=ON`), all entity memory is pre-allocated at compile time, and runtime dynamic creation is not supported.

Configuration file: `~/esp/extra_components/micro_ros_espidf_component/colcon.meta`

- | `RMW_UXRCE_TRANSPORT` | `udp` | UDP transport (Wi-Fi communication) |
- | `RMW_UXRCE_MAX_PUBLISHERS` | `6` | Maximum number of publishers |
- | `RMW_UXRCE_MAX_SUBSCRIPTIONS` | `6` | Maximum number of subscribers |
- | `RMW_UXRCE_MAX_NODES` | `1` | Maximum number of nodes |
- | `RMW_UXRCE_MAX_SERVICES` | `0` | Services not supported |
- | `RMW_UXRCE_MAX_CLIENTS` | `0` | Clients not supported |
- | `RMW_UXRCE_MAX_HISTORY` | `1` | History depth = 1, no message queuing |
- | `UCLIENT_UDP_TRANSPORT_MTU` | `4096` | Maximum UDP packet size 4 KB |
- | `UCLIENT_MIN_HEARTBEAT_TIME_INTERVAL` | `1` | Agent heartbeat interval 1 ms |

Key Constraints:

- `MAX_HISTORY=1` means there is no message buffer queue, so uplink topics (firmware → host) uniformly use **best_effort** QoS
- `MAX_SERVICES=0`, `MAX_CLIENTS=0` means only topic communication is supported, not Service/Client
- After modifying `colcon.meta`, the micro-ROS library must be recompiled for the changes to take effect: run `idf.py clean-microros` in the project directory to clean and rebuild the micro-ROS library, then `idf.py build` to compile the firmware

QoS selection analysis of this experiment:

Direction	Topic	QoS	Reason
Uplink (firmware → host)	<code>/odom_raw</code>	best_effort	<code>MAX_HISTORY=1</code> has no buffering, so reliable retransmission is meaningless; odometry updates at high frequency (10 Hz), losing one frame has little impact
Downlink (host → firmware)	<code>/cmd_vel</code>	reliable	<code>rc1c_subscription_init_default()</code> ; control commands must be reliably delivered, loss may lead to loss of control

The downlink `cmd_vel` uses the default Reliable QoS, relying on `RMW_UXRCE_STREAM_HISTORY=2` (2 MTU buffers) and `RMW_UXRCE_PUBLISH_RELIABLE_TIMEOUT=5` (5 ms timeout) to ensure reliability.

3. Experiment Steps

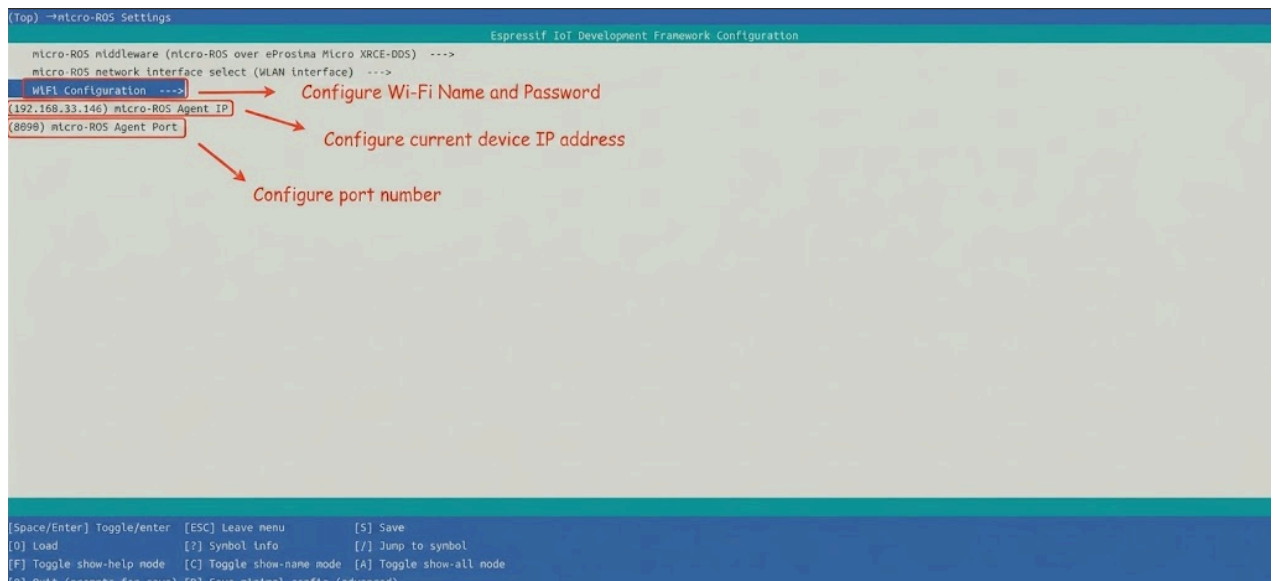
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB Type-C cable	✓ Required
Wi-Fi network (2.4 GHz)	✓ Required
DC geared motor (with encoder) ×2	✓ Required
Motor driver board	✓ Required

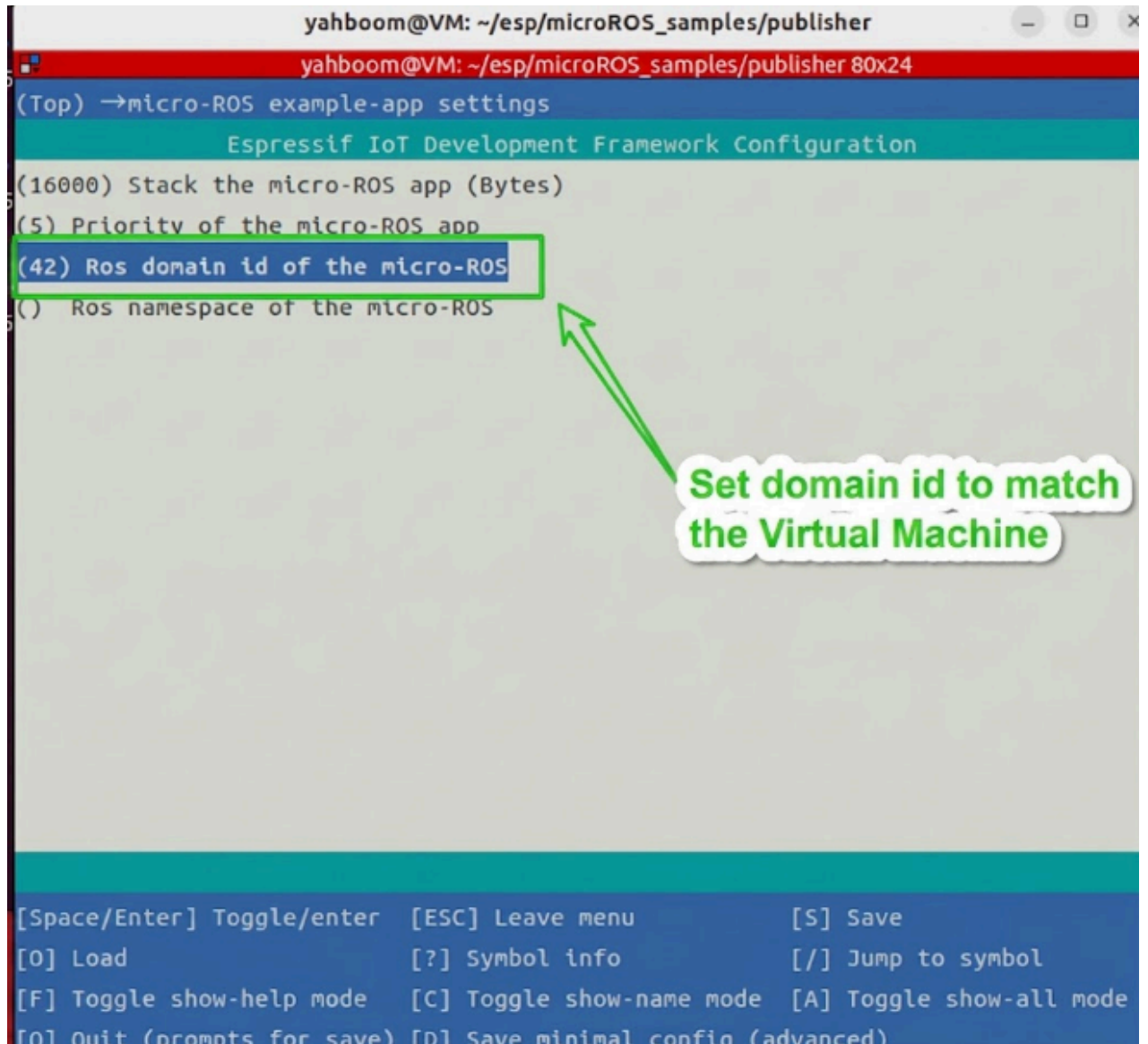
2. Prerequisites

1. The ESP32-IDF development environment has been set up
2. The MicroROS-V2 control board is connected to the computer via USB, and the motors (including encoders) are correctly connected
3. Configure the Wi-Fi SSID, password, Agent IP and port via `idf.py menuconfig` (the `micro_ros` menu item)

```
cd ~/esp/microROS_samples/odom_publisher
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```



4. Select example-app setting to set the domain ID to be consistent with the virtual machine



After configuration, press "s" to save and press "q" to exit the configuration interface.

3. Start micro_ros_agent (Terminal 1)

Start the micro-ROS Agent on the ROS2 host:

```
cd ~/uros_ws
source install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8090 -v4
```

4. Build and Flash (Terminal 2)

```
cd ~/esp/microROS_samples/odom_publisher
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

5. How to Stop

1. First send a stop command: `ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.0}, angular: {z: 0.0}}" -1`
2. Press `Ctrl+J` in Terminal 2 to exit the serial monitor (stop firmware execution)
3. Press `Ctrl+C` in Terminal 1 to stop `micro_ros_agent`

4. Experiment Result

After the build and flash succeed, on the ROS2 host:

View the odometry data:

```
ros2 topic echo /odom_raw
```

Expected output (updated at 10 Hz):

```
header:
  stamp:
    sec: xxx
    nanosec: xxx
  frame_id: odom_frame
child_frame_id: base_footprint
pose:
  pose:
    position:
      x: 0.0
      y: 0.0
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.0
      w: 1.0
  covariance: [...]
twist:
  twist:
    linear:
      x: 0.0
      ...
    angular:
      z: 0.0
      ...
  covariance: [...]
```

Control the car and observe the odometry changes:

```
# Send a forward command
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.3}, angular: {z: 0.0}}"

# Observe the odometry: pose.pose.position.x should keep increasing
ros2 topic echo /odom_raw | grep "x:"
```

Timeout stop protection: If no new `cmd_vel` message is received within 2000 ms (2 seconds), the node automatically sets the motor speed to 0 and stops motion to prevent loss of control. This is the same mechanism as Lesson 6 (`twist_subscriber`, timeout 1500 ms), but with a different timeout threshold.

Control board serial port output:

```
I (xxx) MAIN: Agent reachable: 192.168.x.x:8888
I (xxx) MAIN: free heap before support init: xxx
I (xxx) MAIN: stack watermark before support init: xxx
I (xxx) MAIN: free heap after support init: xxx
I (xxx) MAIN: stack watermark after support init: xxx
I (xxx) MAIN: time synced, offset=xxx ms
I (xxx) MAIN: micro-ROS odom + cmd_vel init done
I (xxx) MAIN: cmd_vel: vx=0.300 wz=0.000
```

5. Program Analysis

Source code paths:

- Firmware source code: `~/esp/microROS_samples/odom_publisher/main/main.c`
- Encoder driver: `~/esp/microROS_samples/odom_publisher/components/encoder/encoder.h`, `encoder.c`

1. Main Program Logic

`main/main.c` is a fully functional chassis control and odometry node, and the flow is as follows:

1. In `app_main()`:

- Calls `Motor_Init()` to initialize the motors and encoders
- Creates the motion queue and the `motion_worker_task` task
- Initializes the Wi-Fi network
- Creates `micro_ros_task`

2. In `micro_ros_task`:

- Creates the node `odom_publisher`
- Initializes static fields of the odometry message such as `frame_id` and covariance matrices (`odom_ros_init`)
- Publishes the topic `odom_raw` in `best_effort` mode, with message type `nav_msgs/msg/Odometry`
- Subscribes to the topic `cmd_vel` in `Reliable` mode, with message type `geometry_msgs/msg/Twist`
- Creates a 100 ms period timer bound to the odometry update callback

- The executor manages 1 timer and 1 subscriber simultaneously (EXECUTOR_HANDLE_COUNT=2)
- Synchronizes the ROS time at startup

2. Key Components

Setting the frame_id of the odometry message:

Source file: `~/esp/microROS_samples/odom_publisher/main/main.c`

```
static void odom_ros_init(void)
{
    const char *content_frame_id = "odom_frame";
    const char *content_child_frame_id = "base_footprint";

    msg_odom.header.frame_id =
        micro_ros_string_utilities_set(msg_odom.header.frame_id, frame_id);
    msg_odom.child_frame_id =
        micro_ros_string_utilities_set(msg_odom.child_frame_id, child_frame_id);

    // Covariance matrix settings
    msg_odom.pose.covariance[0] = 0.001; // x position covariance
    msg_odom.pose.covariance[7] = 0.001; // y position covariance
    msg_odom.pose.covariance[35] = 0.001; // yaw angle covariance
    msg_odom.twist.covariance[0] = 0.0001; // x velocity covariance
    msg_odom.twist.covariance[7] = 0.0001; // y velocity covariance
    msg_odom.twist.covariance[35] = 0.0001; // angular velocity covariance
}
```

Dead Reckoning algorithm:

Source file: `~/esp/microROS_samples/odom_publisher/main/main.c`

```
static void odom_update(float vel_dt, float linear_vel_x,
                       float linear_vel_y, float angular_vel_z)
{
    float delta_heading = angular_vel_z * vel_dt;
    float cos_h = cosf(heading_);
    float sin_h = sinf(heading_);

    // Convert body-frame velocities into world-frame displacement
    float delta_x = (linear_vel_x * cos_h - linear_vel_y * sin_h) * vel_dt;
    float delta_y = (linear_vel_x * sin_h + linear_vel_y * cos_h) * vel_dt;

    x_pos_ += delta_x;
    y_pos_ += delta_y;
    heading_ = normalize_angle(heading_ + delta_heading);

    // Convert Euler angles to a quaternion to fill the orientation
    float q[4];
    odom_euler_to_quat(0.0f, 0.0f, heading_, q);
    msg_odom.pose.pose.orientation.x = (double)q[1];
    msg_odom.pose.pose.orientation.y = (double)q[2];
```



```

msg_odom.pose.pose.orientation.z = (double)q[3];
msg_odom.pose.pose.orientation.w = (double)q[0];

// Fill the velocities
msg_odom.twist.twist.linear.x = linear_vel_x;
msg_odom.twist.twist.angular.z = angular_vel_z;
}

```

`components/encoder/encoder.h` — Encoder interface:

Parameter definition file: `~/esp/microROS_samples/odom_publisher/components/encoder/encoder.h`

Pin/Parameter	Value	Description
<code>ENCODER_GPIO_H1A</code>	5	Phase A of the M1 motor encoder
<code>ENCODER_GPIO_H1B</code>	4	Phase B of the M1 motor encoder
<code>ENCODER_GPIO_H2A</code>	7	Phase A of the M2 motor encoder
<code>ENCODER_GPIO_H2B</code>	6	Phase B of the M2 motor encoder
<code>ENCODER_PCNT_HIGH_LIMIT</code>	1000	Pulse counter upper limit
<code>ENCODER_PCNT_LOW_LIMIT</code>	-1000	Pulse counter lower limit

Function	Description
<code>Encoder_Init()</code>	Initializes the PCNT peripherals of the two encoders
<code>Encoder_Get_Count_M1()</code>	Gets the count value of the M1 motor encoder
<code>Encoder_Get_Count_M2()</code>	Gets the count value of the M2 motor encoder

ROS time synchronization mechanism:

Source file: `~/esp/microROS_samples/odom_publisher/main/main.c`

```

static void sync_time(void)
{
    if (RMW_RET_OK == rmw_uros_sync_session(1000)) {
        uint64_t now_ms = get_millisecond();
        uint64_t ros_time_ms = rmw_uros_epoch_millis();
        time_offset_ms = ros_time_ms - now_ms;
        ESP_LOGI(TAG, "time synced, offset=%llu ms",
                 (unsigned long long)time_offset_ms);
    } else {
        time_offset_ms = 0;
        ESP_LOGW(TAG, "time sync failed, use local uptime");
    }
}

static struct timespec get_timespec(void)

```

```
{
    struct timespec tp = {0};
    uint64_t now_ms = get_millisecond() + time_offset_ms;
    tp.tv_sec = (time_t)(now_ms / 1000ULL);
    tp.tv_nsec = (long)((now_ms % 1000ULL) * 1000000ULL);
    return tp;
}
```

Timer callback `timer_odom_callback`:

Source file: `~/esp/microROS_samples/odom_publisher/main/main.c`

Source file: `~/esp/microROS_samples/odom_publisher/main/main.c`

```
static void timer_odom_callback(rcl_timer_t *timer, int64_t last_call_time)
{
    RCLC_UNUSED(last_call_time);

    if (timer == NULL) {
        return;
    }

    uint64_t now_ms = get_millisecond();
    if (prev_odom_update_ms == 0) {
        prev_odom_update_ms = now_ms;
    }

    // 1. Calculate the time interval
    float vel_dt = (float)(now_ms - prev_odom_update_ms) / 1000.0f;
    prev_odom_update_ms = now_ms;

    // 2. Get the actual speed estimated from the encoders
    Motion_Get_Speed(&car_motion);

    // 3. Dead Reckoning update
    odom_update(
        vel_dt,
        car_motion.Vx,
        car_motion.Vy,
        car_motion.Wz);

    // 4. Fill the ROS timestamp
    struct timespec time_stamp = get_timespec();
    msg_odom.header.stamp.sec = (int32_t)time_stamp.tv_sec;
    msg_odom.header.stamp.nanosec = (uint32_t)time_stamp.tv_nsec;

    // 5. Publish the odometry message
    rcl_ret_t rc = rcl_publish(&publisher_odom, &msg_odom, NULL);
    if (rc != RCL_RET_OK) {
        odom_pub_fail_cnt++;
        if ((odom_pub_fail_cnt % 20U) == 1U) {
            print_rcl_error("rcl_publish(&publisher_odom, &msg_odom, NULL)",
                __LINE__, rc);
        }
    }
}
```

```
}
```

6. Common Problems

Problem	Cause	Solution
Odometry position data does not update	The encoders are not read correctly or the motors are not rotating	Check the encoder wiring and confirm the motors are actually rotating
Severe odometry data drift	Dead Reckoning error accumulation	This is an inherent limitation of the open-loop node; fuse IMU or external localization to correct it
Inaccurate timestamps	ROS time synchronization failed	Check whether the Agent time synchronization function is normal; it will automatically attempt synchronization at startup
frame_id mismatch	Inconsistent coordinate frame conventions with other ROS2 nodes	The default is <code>odom_frame</code> → <code>base_footprint</code> ; modify as needed
Inaccurate encoder counts	Pulse count overflow or incorrect direction judgment	Adjust the <code>ENCODER_PCNT_HIGH_LIMIT</code> and <code>MOTOR_ENCODER_SIGN</code> macros